MODULUS

Retail Operations Platform

API Contract Document

Version 1.0

1. Overview

This document defines all API endpoints consumed by the Modulus platform. All endpoints are served by Mock Service Worker (MSW) running in the browser. There is no backend server. MSW intercepts fetch requests at the defined paths and returns seeded JSON responses that conform to the type definitions in packages/types.

All endpoints follow REST conventions. All request and response bodies use JSON. All timestamps are ISO 8601 strings. All monetary values are numbers representing currency in the smallest unit (cents).

2. Authentication

All endpoints except POST /api/auth/login require a valid Bearer token in the Authorization header.

```
Authorization: Bearer <jwt_token>
```

Requests without a valid token receive a 401 response. MSW validates the token format but does not perform cryptographic verification. Any well-formed JWT string is accepted in the mock environment.

POST **/api/auth/login** Authenticate and receive a JWT token

Request body:

```
{
  "email": "ops@modulus.com",
  "password": "password123"
}
```

Success response (200):

```
{
  "token": "<jwt_string>",
  "user": {
    "id": "u1",
    "email": "ops@modulus.com",
    "name": "Operations Manager",
    "role": "ops_manager"
  }
}
```

Error response (401):

```
{ "error": "Invalid credentials" }
```

3. Products API

All endpoints in this section are consumed exclusively by modulus-inventory. The base path is /api/products.

GET **/api/products** Return a paginated list of products

Query parameters:

Parameter	Type	Required	Description
page	number	No (default: 1)	Page number, 1-indexed
limit	number	No (default: 20)	Items per page, maximum 100
search	string	No	Case-insensitive match against name or SKU
category	string	No	Filter by category name
stockStatus	string	No	One of: in_stock, low_stock, out_of_stock
active	boolean	No (default: true)	Filter by active status

Success response (200):

```
{
  "data": [{ "id": "p1", "name": "...", "sku": "...", "category": "...",
             "price": 2999, "stock": 45, "active": true, "imageUrl": "..." }],
  "pagination": { "page": 1, "limit": 20, "total": 120, "totalPages": 6 }
}
```

POST **/api/products** Create a new product

Request body:

```
{
  "name": "Wireless Headphones",
  "sku": "WH-1001",
  "category": "Electronics",
  "price": 7999,
  "stock": 50,
```

```
  "imageUrl": "https://..."  // optional
}
```

Success response (201): the created Product object.

Error responses: 400 if validation fails, 409 if SKU already exists.

GET **/api/products/:id** Return a single product by ID

Success response (200): the Product object. Error response (404) if not found.

PATCH **/api/products/:id** Update product fields

Request body: any subset of Product fields except id and sku. Only provided fields are updated.

Success response (200): the updated Product object. Error response (404) if not found.

PATCH **/api/products/:id/status** Toggle product active status

Request body:

```
{ "active": false }
```

Success response (200): the updated Product object with the new active value.

4. Categories API

GET **/api/categories** Return all categories

Success response (200):

```
{ "data": [{ "id": "c1", "name": "Electronics" }, ...] }
```

POST **/api/categories** Create a new category

```
{ "name": "New Category" }
```

Success response (201): the created Category object. Error response (409) if name already exists.

PATCH **/api/categories/:id** Rename a category

```
{ "name": "Renamed Category" }
```

Success response (200): the updated Category object. All products in this category are updated to the new name.

5. Orders API

All endpoints in this section are consumed exclusively by modulus-orders.

GET **/api/orders** Return a paginated list of orders

Query parameters:

Parameter	Type	Required	Description
page	number	No (default: 1)	Page number
limit	number	No (default: 20)	Items per page
status	string	No	One of: pending, processing, shipped, delivered, cancelled
search	string	No	Match against order ID or customer email
customerId	string	No	Filter by customer ID

Success response (200):

```
{
  "data": [{
    "id": "o1", "customerId": "c1", "customerEmail": "alice@...",
    "status": "pending", "items": [...], "total": 14999,
    "createdAt": "2025-03-01T10:00:00Z",
    "timeline": [{ "status": "pending", "timestamp": "2025-03-01T10:00:00Z" }]
  }],
  "pagination": { "page": 1, "limit": 20, "total": 500, "totalPages": 25 }
}
```

GET **/api/orders/:id** Return a single order by ID

Success response (200): the full Order object including all items and timeline. Error response (404) if not found.

PATCH **/api/orders/:id/status** Update order status

Request body:

```
{ "status": "processing" }
```

Validation: the requested transition must be valid per the state machine. Valid transitions:

Current Status	Valid Next Statuses
pending	processing, cancelled
processing	shipped, cancelled
shipped	delivered, cancelled
delivered	None (terminal state)
cancelled	None (terminal state)

Success response (200): the updated Order object with the new status appended to timeline.

Error response (422) if the requested transition is invalid, with body:

```
{ "error": "Invalid transition", "from": "delivered", "to": "processing" }
```

6. Customers API

GET **/api/customers/:id** Return a customer by ID

Success response (200):

```
{ "id": "c1", "email": "alice@example.com", "name": "Alice Chen",
"totalOrders": 8 }
```

GET **/api/customers** Search customers by email

Query parameter: email (required). Returns all customers whose email contains the search string.

Success response (200):

```
{ "data": [{ "id": "c1", "email": "alice@example.com", "name": "Alice Chen",
"totalOrders": 8 }] }
```

7. Analytics API

All endpoints in this section are consumed exclusively by modulus-analytics. All data is pre-aggregated in the MSW seed. No computation happens at request time.

GET **/api/analytics/summary** Return top-level metric cards

Query parameters: startDate and endDate (ISO 8601 date strings, both required).

Success response (200):

```
{
  "totalRevenue": 4823600,
  "totalOrders": 482,
  "averageOrderValue": 10007,
  "returnRate": 0.034
}
```

GET **/api/analytics/revenue** Return daily revenue for a date range

Query parameters: startDate and endDate (ISO 8601 date strings, both required).

Success response (200):

```
{ "data": [{ "date": "2025-03-01", "revenue": 182400 }, ...] }
```

GET **/api/analytics/orders-by-status** Return order count grouped by status

Query parameters: startDate and endDate.

Success response (200):

```
{ "data": [{ "status": "delivered", "count": 210 }, ...] }
```

GET **/api/analytics/top-products** Return top 10 products by revenue

Query parameters: startDate and endDate.

Success response (200):

```
{ "data": [{ "productId": "p12", "name": "...", "revenue": 284000, "unitsSold":
142 }, ...] }
```

GET **/api/analytics/regional** Return sales breakdown by region

Query parameters: startDate and endDate.

Success response (200):

```
{ "data": [{ "region": "Northeast", "revenue": 1240000, "orders": 124 }, ...] }
```

8. Standard Error Responses

All endpoints return errors in the following format:

```
{ "error": "<human-readable message>", "code": "<machine-readable code>" }
```

HTTP Status	Code	Meaning
400	VALIDATION_ERROR	Request body failed schema validation
401	UNAUTHORIZED	Missing or malformed Authorization header
403	FORBIDDEN	Token valid but role lacks permission for this resource
404	NOT_FOUND	Requested resource does not exist
409	CONFLICT	Resource already exists (duplicate SKU, duplicate category name)
422	INVALID_TRANSITION	Requested state machine transition is not permitted
500	INTERNAL_ERROR	Unexpected MSW handler error

9. MSW Handler Implementation Notes

- All handlers are defined in packages/types/src/handlers.ts and imported by the shell for MSW initialisation.
- Handler response shapes must match the TypeScript types defined in packages/types/src/types.ts exactly. A type change without a corresponding handler update is a compile error.
- Simulated network latency of 300ms to 600ms is applied to all handlers to produce realistic loading states.
- The seed data is generated once at MSW startup and held in module-level variables. Mutations (POST, PATCH) update the in-memory seed. Data resets on page refresh.
- Pagination is performed in the handler, not in the seed data. The handler slices the full array based on page and limit parameters.
- Error simulation: adding the request header X-Simulate-Error: true to any request causes the handler to return a 500 response. This is used in E2E tests to verify error state handling.